SPATIAL MAPPING USING TIME-OF-FLIGHT

COMPENG 2DX3: Final Report

Mohammad Hadi, hadim4, 400383319

Professors: Dr. Athar, Dr. Doyle, Dr. Haddara

04/13/2025

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is our own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario. [Mohammad Hadi, 40038319]

Device Overview – Features

This system offers an exceptionally user-friendly experience. An onboard button will simply start the device. Followed by 4 onboard LEDs that allows the user to know its current state and what its doing (ie. If it has started, if it's currently scanning, or its direction).

The Texas Instrument MSP-ESP432E401Y Microcontroller Features:

- 32-Bit ARM Cortex-M4F CPU
- 1024 kb Flash Memory, 256 kb SRAM, 6 kb EEPROM
- Up to 120 MHz Bus Speed [Code customized to 12 mHz]
- 4x Onboard LEDs
- 2x Onboard Buttons
- GPIO Pins for all external components

The VL53LIX Time of Flight Sensor Features:

- Scans up to 4 Metres
- Operating Voltage between 2.6V-5V
- Ranging Frequency up to 50Hz

The 28BYJ-48 Stepper Motor Features:

- 512 Steps for a full 360-degree rotation/ 1 Revolution
- 4 Phases per step
- 4 Onboard LEDs which show current phase
- Supply Voltage: 5 – 12 V

Serial Communication:

- I2C Protocol is used for ToF Sensor to send data to MCU
- UART Protocol is used for MCU to send data to PC
- A baud rate of 115200 BPS is used for UART

Visualization:

- Python is used for UART communication between the MCU and PC
    - You can use Juypter or other IDEs
- Open3D is used within python for data visualization and 3D model generation with the use of NumPy

Components	Price $(CAD)
MSP-ESP432E401Y Microcontroller	$70
The VL53LIX Time of Flight Sensor	$30
28BYJ-48 Stepper Motor with driver	$6

Table 1: Cost of hardware [Price is an approximation]

Device Overview - General Description

This device provides the user with an efficient, cost-effective solution for performing various scans within a room, enabling a digitalized 3D visualization of the room. Essentially, with the press of PJ1, the system will initialize and begin to take distance measurements at every 11.25 degrees along the yz-plane, across any set number of full revolutions with the implementation of the 28BYJ-48 stepper motor and VL53LIX ToF sensor. The ToF sensor is responsible for capturing distance measurements at the specified angle. The VL53LIX is an infrared scanner, so it does this by emitting light then receiving it and measuring the time it took for the light to bounce back and be received. To specify, the sensor uses this formula to determine the distance: Distance = ($\frac{Flight\ Time}{2}$) ∗(Speed of Light). Every time a measurement is captured, PN1 will blink, ensuring this step was successful. These measurements are then sent to the MSP-ESP432E401Y microcontroller through I2C. Moreover, with the use of UART (through USB), the microcontroller then transmits this data to the PC. Once the data is received, it is turned into cartesian coordinates using trigonometry. All essential data points are being written into a separate folder named “ToF_Radar.xyz” and printed live. Finally, the program plots all the xyz coordinates using NumPy. Lastly, everything is visualized in 3D using the open3D python library, which then presents us with a new tab showcasing the 3D model.

Block Diagram

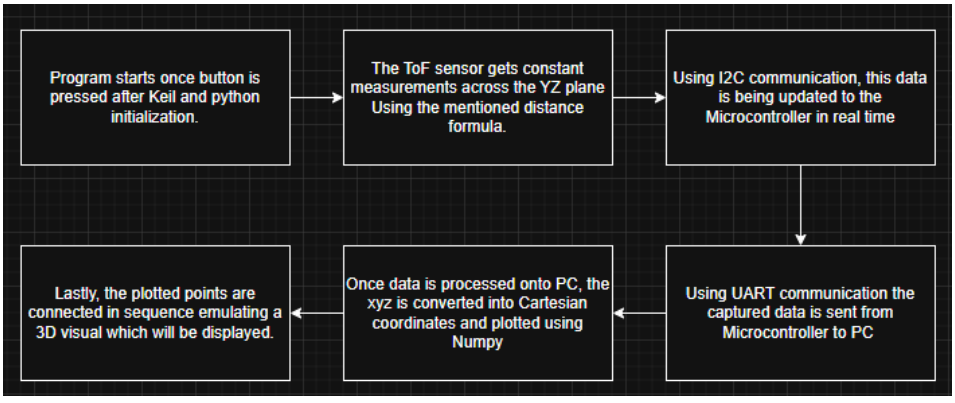


Figure 1: Block Diagram

Device Characteristics

Table 2: Pin assignment

MSP432E01Y Microcontroller		**28BYJ-48 Stepper Motor**		**VL53LIX Time of Flight Sensor**	
Bus Speed	12 MHz	**V+**	5 V	**Vin**	3.3 V
Measurement LED	PN1	**V-**	GND	**GND**	GND
Status LED	PN0	**IN1**	PH0	**SDA**	PB3
Uart Tx LED	PF4	**IN2**	PH1	**SCL**	PB2
On-Board Button	PJ1	**IN3**	PH2		
Serial Port	COM4	**IN4**	PH3		
Baud Rate	115200 BPS				

Detailed Description – Distance Measurement Acquisition

The VL53LIX ToF Sensor essentially works by emitting a beam of light (Infarred) using the emitter. The sensor scans up to 4 meters. Once the light hits an object, it bounces back to the receiver. The emitter and receiver are the 2 shiny rectangles on the Sensor. The ToF sensor calculates the distance with this formula: Distance = ($\frac{Flight\ Time}{2}$) ∗(Speed of Light). The distance is taken in terms of Millimeters. Furthermore, the measurements are then transmitted to the Microcontroller using serial communication, via I2C. This process is done repeatedly along a YZ axis 32 times per every revolution. The number of revolutions is dependent on the user. In the base code, the system scans at every 11.25 degrees (32 times per 360 degrees), for a total of 3 times.

Additionally, the raw distance data captured from the ToF sensor is processed by the system. Every measurement taken will then be converted to Y and Z using the following trigonometric formulas.

$Y = Distance * \sin\theta \mid Z = Distance * \cos\theta$

In this formula the D is every measurement captured in mm. The angle is incremented by itself every time distance is captured. For example, the angle will begin at 11.25 degrees and multiply itself by 2 every measurement within the 32 total. After a full scan, the angle will reset, and the procedure will repeat itself for the next scan, until the set total # of scans condition is met. The x coordinate is implemented by the user. The user must manually move in between each scan. You must determine the physical distance between each scan and update that number in terms of mm onto python. An example of the trig can be seen in Figure 2.

```
1 , angle:  11.25 distance:  347.0 |     x= 0 y=  67.6963417395965 z=  340.33249229992094
2 , angle:  22.5 distance:  139.0 |     x= 0 y=  53.19299709874748 z=  128.41925501906886
```

Figure 2: Trig example

Detailed Description – Visualization

In order to visualize the processed data, we begin by using the PySerial library on python which utilizes UART at a baud rate of 115200 BPS and a timeout of 30 seconds. Receiving data as bytes, which is then turned into floats. As mentioned prior, we use the math library in python to convert the distance into cartesian coordinates.

After the program gets all the (x,y,z) coordinates for every scan, it uses NumPy to plot the data points across a 3D axis. We lastly utilize the Open3D library to connect the points. A mesh like visual is then created using a nested for loop which connects each of the 32 points, then connects the points from layers. By looking at Figure 3 and 4, you can see how the coordinated are placed initially, then connected by lines for a full visualization.



Figure 3: coordinates displaced

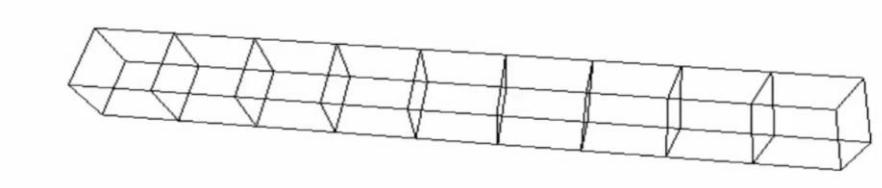


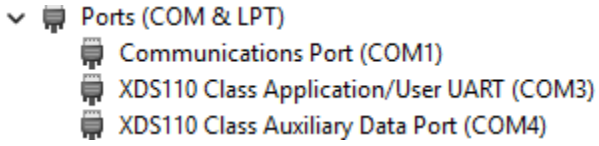
Figure 4: coordinates connected

Application Note

In order to use this device, it is assumed that the user currently has the following applications installed and configured.

- *Keil uVision 5*
- *Python [3.7 - 3.10] – Along with the following libraries*
  - *Math*
  - *PySerial*
  - *Open3D*
  - NumPy

1. Connect a USB to the USB port of your microcontroller, then connect it to your PC. A yellow light below the USB port will turn on – meaning the microcontroller is turned on.
2. You must identify your COM port to use serial communication. To do this, open the device manager on your PC and click on "Ports (COM & LPT)". You will see "XDS110 Class Application/ User UART. In this example, for my PC COM3 is required, however it might be different for your computer.



3. After figuring out your COM#, you will open the python program and initialize it in this line as shown. In this example I am using Juypter.

```
s = serial.Serial("COM3", 115200, timeout=30
```

4. Now you must configure the physical hardware pin layout onto your microcontroller. Refer to Table 2 to connect each wire and inputs to their corresponding pin.
5. It is important to note that in order for the device to complete a set number of revolutions you must locate the below for loop on Keil and change the number "3" to your choosing.

Then on Python change the range of the for loop (# of scans per revolution * your desired # of scans), currently it is 96 (32*3). Lastly, update this part off the code with the prior information as well.

```
// Motor control and distance measurement
if (motor) {
  for (int i = 0; i < 3; i++) {
      counter = 0;
      steps = 0;
      dir = 1;
```

```
for i in range(96):
    counter += 1
    angle_1 += 11.25
    distance_S = s.readline().decode().strip()
    distance = float(distance_S)
```

```
scan = 3
num  = 32
```

6. Once done. Open the project on Keil. Translate, build, and load the code onto the microcontroller. Afterward, click the reset button which is in series with the USB port.
7. Open Python and run the code. You will be prompted to press “Enter”. afterward, press PJ1 and the motor will begin rotating CCW while python updates each measured distance along with the # of scan and XYZ coordinates.

```
Jupyter environment detected. Enabling Open3D WebVisualizer.
[Open3D INFO] WebRTC GUI backend enabled.
[Open3D INFO] WebRTCWindowSystem: HTTP handshake server disabled.
Opening: COM3
Press Enter to start communication...
1 , angle:  11.25 distance:  347.0 |    x= 0 y=  67.6963417395965 z=  340.33249229992094
2 , angle:  22.5 distance:  139.0 |    x= 0 y=  53.19299709874748 z=  128.41925501906886
3 , angle:  33.75 distance:  81.0 |    x= 0 y=  45.00118874587776 z=  67.34903859650616
4 , angle:  45.0 distance:  63.0 |    x= 0 y=  44.5477272147525 z=  44.5477272147525
5 , angle:  56.25 distance:  67.0 |    x= 0 y=  55.70846402427053 z=  37.223205612313335
6 , angle:  67.5 distance:  68.0 |    x= 0 y=  62.823808210767496 z=  26.022473400826108
7 , angle:  78.75 distance:  67.0 |    x= 0 y=  65.71261378701644 z=  13.071051575000599
8 , angle:  90.0 distance:  71.0 |    x= 0 y=  71.0 z=  4.347496136973104e-15
9 , angle:  101.25 distance:  77.0 |    x= 0 y=  75.52046659104874 z=  -15.021954795241871
10 , angle:  112.5 distance:  84.0 |    x= 0 y=  77.60588073094809 z=  -32.145408318667535
11 , angle:  123.75 distance:  89.0 |    x= 0 y=  74.00079549492852 z=  -49.44575073874461
12 , angle:  135.0 distance:  141.0 |    x= 0 y=  99.70285614730321 z=  -99.70285614730321
13 , angle:  146.25 distance:  839.0 |    x= 0 y=  466.12342550344624 z=  -697.6030047218355
14 , angle:  157.5 distance:  988.0 |    x= 0 y=  378.09123111767088 z=  -912.7929701211514
15 , angle:  168.75 distance:  994.0 |    x= 0 y=  193.91978000440314 z=  -974.9005687208111
16 , angle:  180.0 distance:  1085.0 |    x= 0 y=  1.3287417777074878e-13 z=  -1085.0
17 , angle:  191.25 distance:  1135.0 |    x= 0 y=  -221.42751548830057 z=  -1113.1912932576665
18 , angle:  202.5 distance:  1293.0 |    x= 0 y=  -494.80967804806096 z=  -1194.576235537094
19 , angle:  213.75 distance:  1698.0 |    x= 0 y=  -943.3582556672847 z=  -1411.8354016897217
20 , angle:  225.0 distance:  1730.0 |    x= 0 y=  -1223.294731452727 z=  -1223.2947314527275
21 , angle:  236.25 distance:  1562.0 |    x= 0 y=  -1298.7555344165758 z=  -867.8007039766186
22 , angle:  247.5 distance:  1503.0 |    x= 0 y=  -1388.5909373644644 z=  -575.1731988447295
23 , angle:  258.75 distance:  1494.0 |    x= 0 y=  -1405.2932089224262 z=  -291.4649410920962
24 , angle:  270.0 distance:  1527.0 |    x= 0 y=  -1527.0 z=  -2.8050534934470l2e-13
25 , angle:  281.25 distance:  870.0 |    x= 0 y=  -853.2831939508105 z=  169.72858015403162
26 , angle:  292.5 distance:  600.0 |    x= 0 y=  -554.3277195067719 z=  229.610059419054
27 , angle:  303.75 distance:  597.0 |    x= 0 y=  -496.38735854461964 z=  331.6754291127023
28 , angle:  315.0 distance:  722.0 |    x= 0 y=  -510.5310960166874 z=  510.5310960166872
29 , angle:  326.25 distance:  972.0 |    x= 0 y=  -540.0142664950533 z=  808.188463158074
30 , angle:  337.5 distance:  1016.0 |    x= 0 y=  -694.9531131750026 z=  1677.765231040497
31 , angle:  348.75 distance:  1890.0 |    x= 0 y=  -368.7207086104832l z=  1853.6041799621052
32 , angle:  360.0 distance:  1492.0 |    x= 0 y=  -3.6543460486557026-13 z=  1492.0

 NEXT SCAN

1 , angle:  11.25 distance:  757.0 |    x= 1300 y=  147.68337376620091 z=  742.4544572652454
2 , angle:  22.5 distance:  100.0 |    x= 1300 y=  38.268343236508976 z=  92.38795325112868
3 , angle:  33.75 distance:  76.0 |    x= 1300 y=  42.223337709480077 z=  63.191609053499344
4 , angle:  45.0 distance:  69.0 |    x= 1300 y=  48.790367908718 z=  48.790367908718178
5 , angle:  56.25 distance:  67.0 |    x= 1300 y=  55.70846402427053 z=  37.223205612313335
6 , angle:  67.5 distance:  68.0 |    x= 1300 y=  62.823808210767496 z=  26.022473400826108
```

8. The code is designed to scan every 11.25 degrees along a 360-degree rotation. The motor is capturing distances along the YZ plane. Once the first scan full scan (32 distances) is done, the motor will rotate CW back to its original position. This is done in order to untangle the wires. while the device is rotating CW D4 will light on, which is the user’s que to move forward in the X direction in order for it to update. [You are the X displacement. Locate the x_inc variable within python and update the distance between

each in mm. The device will assume you have moved in the x direction dependant on this variable.

```
x_inc = 4500
```

9. While scanning, you must consider that the sensor will not properly be able to scan if presented with glass.

10. If all steps are followed correctly and all scans are done, another tab will open showcasing the captured 3D model. You will also be able to interact with the visual.

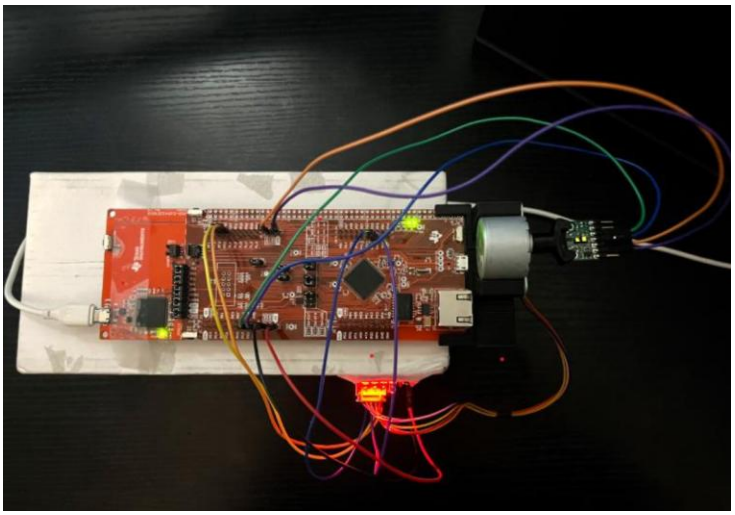


Figure 5: Physical implementation

Figure 3 depicts a fully assembled implementation of the LiDAR Spatial Mapping System. If all steps are followed correctly, you should have something like this.

Expected Output

An expected output can be seen in Figure 6. This scan is on the second floor of JHE, which was the room I was given to scan. You can compare the 3D visual and actual location side by side. On the middle left, the visual pulls itself toward me because there was glass there, however, besides that you can see that this device is accurate.

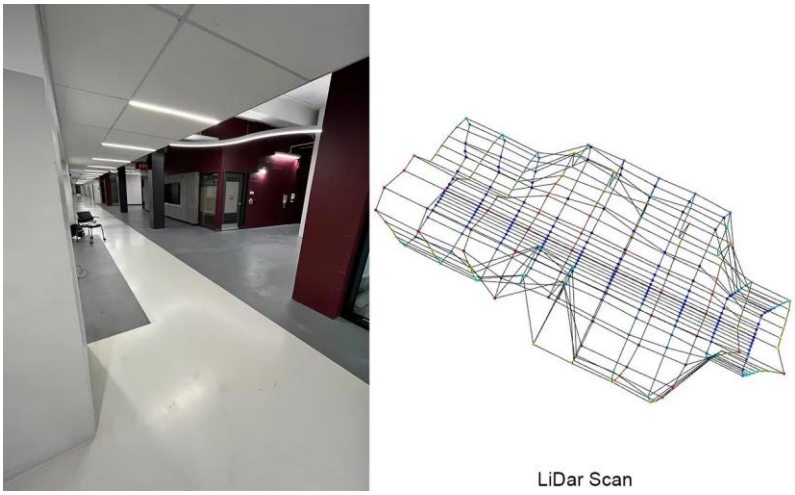


Figure 6: 3D scan compared to physical location.

Limitations

Limitations of the microcontroller

The MSP432E401Y utilizes a 32-bit floating point unit (FPU) allowing the embedded system to support single precision floating point operations. This is decent, however, can result in some limitations, especially with the trigonometric calculations. The problem arises when handling double. In terms of this device this issue can lead to rounding errors as well as loss of precision. This will have an impact on the calculated cartesian coordinates over time.

Maximum quantization error

The ToF sensor has a 1mm resolution. Therefore, meaning the Maximum Quantization error is 1mm. This is fine for most indoor scans, such as mine in figure 6. However it can result in minor distortions when more detail is required.

What is the maximum standard serial communication rate?

The maximum standard serial communication rate that can be implemented is 128000 BPS. In order to verify this, we open device manager and view the port settings under XDS110 Class Application/ User Uart. This is seen in figure 5.

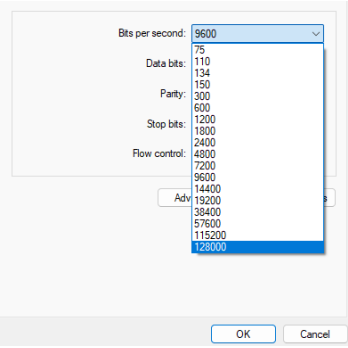


Figure 7: Max BPS

What were the communication method(s) and speed used?

In order to send data from the ToF sensor, the communication protocol used is I2C. The speed used was 100 kbps on standard mode.

P***rimary limitations on speed***

One of the primary limitations on the system was the speed of the stepper motor. After every step in the motor rotation, there was always a delay of 5. The reason I changed the delay from 1 to 5 was because I was assigned a Bus Speed of 12 mHz. This was really fast and through testing I noticed that the motor was not even rotating. However, after changing the delay to 5 the motor worked. The system is negatively impacted by the speed of the motor.

```
153  //CLOWKWISE ROTATION
154  void CW(){
155    uint32_t delay = 5;
156
157      GPIO_PORTH_DATA_R = 0b1001;
158      SysTick_Wait10ms(delay);
159      GPIO_PORTH_DATA_R = 0b0011;
160      SysTick_Wait10ms(delay);
161      GPIO_PORTH_DATA_R = 0b0110;
162      SysTick_Wait10ms(delay);
163      GPIO_PORTH_DATA_R = 0b1100;
164      SysTick_Wait10ms(delay);
165  }
166
167  //COUNTERCLOCKWISE ROTATION
168  void CCW(){
169    uint32_t delay = 5;
170
171      GPIO_PORTH_DATA_R = 0b1100;
172      SysTick_Wait10ms(delay);
173      GPIO_PORTH_DATA_R = 0b0110;
174      SysTick_Wait10ms(delay);
175      GPIO_PORTH_DATA_R = 0b0011;
176      SysTick_Wait10ms(delay);
177      GPIO_PORTH_DATA_R = 0b1001;
178      SysTick_Wait10ms(delay);
179  }
```

Figure 8: Step delay

Moving forward with the ToF sensor, its maximum ranging frequency is 50 Hz, meaning its somewhat restricted when rapidly transmitting measurements. Ultimately, comparing both these devices, it is noticed that the ToF sensor is this embedded systems primary bottleneck as it slows down the system as a whole.

Circuit Schematic

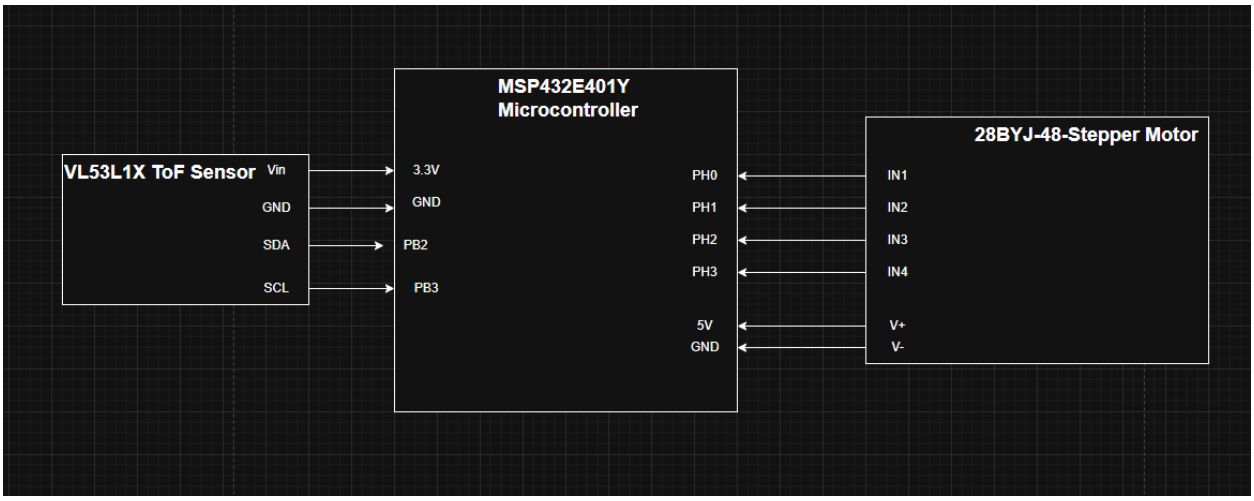


Figure 9: Circuit Schematic

Programming Logic

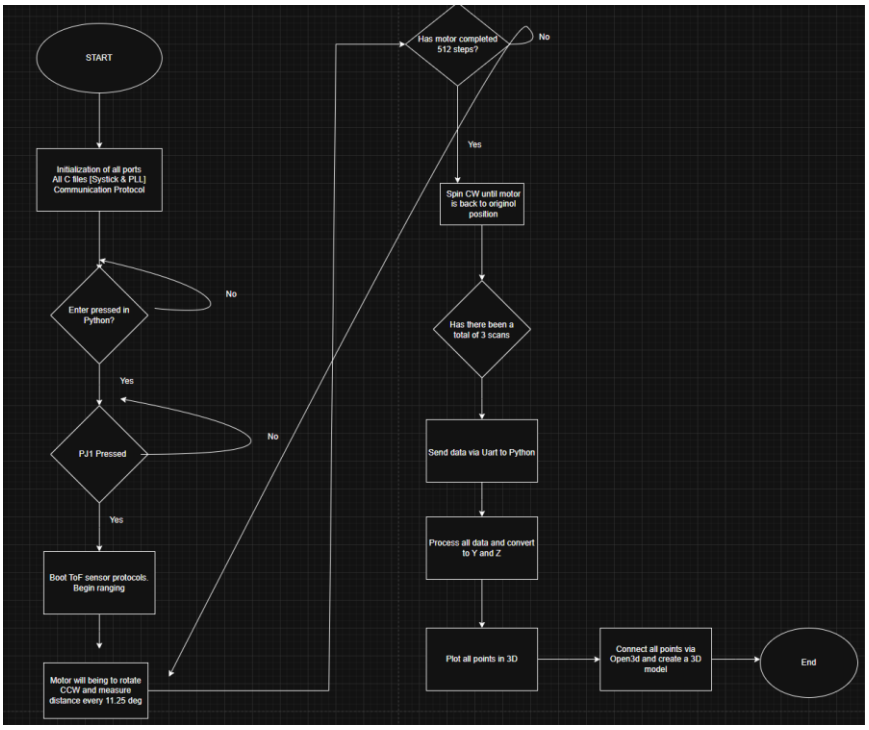


Figure 10: Flowchart